AI, V2X & Graph Neural Networks

for Traffic Systems

A Hands-On Workshop

National Engineering School of Sousse, Tunisia

Workshop Overview

What you will learn today:

- How AI is transforming transportation
- What V2X communication means and why it matters
- How to represent a **vehicle network** as a **graph**
- How **Graph Neural Networks** learn from vehicle sensor data
- Hands-on: build and train a GNN on V2X-Sim-2.0 data

The Problem We're Solving

> Given a network of connected vehicles, each equipped with multiple sensors, can we **predict their future behavior**?

Input: Sensor feature vectors from each vehicle (position, speed, heading, acceleration, LiDAR, camera)

Output: Predicted next speed / risk of collision

Why it matters:

- Reduce commute times and accidents
- Enable autonomous driving decisions
- Power cooperative vehicle intelligence
- Build smarter V2X infrastructure

Part 1

AI in Automotive & V2X

AI in Automotive

AI is driving the future of transportation:

- **Self-driving cars** — perception, planning, control
- **Traffic management** — adaptive signals, flow optimization
- **Predictive maintenance** — fleet management
- **Route optimization** — real-time navigation
- **Safety systems** — collision avoidance, driver monitoring

The automotive industry generates **massive amounts of sensor data** — AI turns that data into decisions.

What is V2X?

V2X = Vehicle-to-Everything

A family of communication technologies:

Type	Meaning	Example
V2V	Vehicle-to-Vehicle	Cars sharing brake warnings
V2I	Vehicle-to-Infrastructure	Traffic lights sending timing info
V2P	Vehicle-to-Pedestrian	Alerting nearby pedestrians
V2N	Vehicle-to-Network	Cloud-based traffic updates

Why V2X Matters

Without V2X: Each vehicle sees only its own sensors

With V2X: Vehicles share sensor data across the network

Benefits:

- **Safety:** Warn about hazards beyond line of sight
- **Efficiency:** Coordinate speeds to reduce congestion
- **Prediction:** More sensor data = better behavior forecasting
- **Autonomy:** Essential for self-driving at scale

> V2X transforms isolated vehicles into a **connected intelligent network**

Real-World Motivation

Traffic congestion costs:

- Billions of dollars in lost productivity annually
- Increased fuel consumption and emissions
- Higher accident rates

The opportunity:

- Predict vehicle behavior **before** problems happen
- Coordinate vehicles cooperatively
- Enable proactive, not reactive, traffic management
- Use shared sensor data for collective intelligence

Part 2

Why Graphs?

The Limits of Traditional AI for Vehicles

CNNs (Convolutional Neural Networks):

- Great for images (grids of pixels)
- Vehicle networks are NOT a grid — they're irregular
- CNNs can't naturally capture vehicle-to-vehicle topology

RNNs (Recurrent Neural Networks):

- Great for sequences (time series)
- Handle temporal patterns well
- But ignore **spatial relationships** between vehicles

> We need a model that understands **both** the network structure **and** each vehicle's sensor data

A Vehicle Network IS a Graph

Think of vehicles on a road:

- **Nodes** = vehicles
- **Edges** = proximity or communication links between them
- **Node features** = each vehicle's sensor vector
- **Edge weights** = distance between vehicles

```
[Car A] ----80m---- [Car B]
   |                    |
 60m                  70m
   |                    |
[Car C] ----50m---- [Car D]
```

> Graphs naturally represent the **dynamic topology** of vehicle networks

Vehicle Sensor Feature Vector

Each vehicle node carries a **rich feature vector** from its onboard sensors:

Sensor	Feature	Unit
GPS	x, y position	meters
Speedometer	speed	m/s
IMU	heading, acceleration	rad, m/s^2
LiDAR	range to nearest object	meters
Camera	detection confidence	0.0 - 1.0

$$feature_vector = [x, y, speed, heading, accel, lidar_range, cam_conf]$$

Why Graphs Are Perfect for Vehicles

- Nearby vehicles **influence** each other's behavior
  - If Car A brakes suddenly, Car B behind it must react
- The influence follows the **graph structure** (proximity links)
- Graph-based models can learn these spatial dependencies

Key insight:

> A vehicle's future behavior depends not just on its own sensors, but on **what nearby vehicles are doing**

Part 3

Introduction to Graph Neural Networks

What is a GNN? (Intuition)

Imagine you're driving and want to predict what happens next:

- You check **your own** sensors (speed, GPS, camera)
- You also receive data from **nearby vehicles** (V2X messages)
- You **combine** your data with theirs to make a better prediction

That's exactly what a GNN does!

Each vehicle node:

1. Collects sensor data from its **neighbors**
2. **Aggregates** it (average, sum, etc.)
3. **Updates** its own representation

Message Passing — The Core Idea

GNNs work through **message passing** — like V2X communication:

```
      [Car A]                      [Car A]
     features:                   receives from
   [x,y,spd,hdg]               B and C:
       / \          ------>        / \
      /   \                       /   \
  [Car B] [Car C]           [Car B] [Car C]
   sends    sends
  its data  its data

 Step 1: SEND          Step 2: AGGREGATE      Step 3: UPDATE
 Each vehicle           Each vehicle            Each vehicle
 broadcasts its  --->   collects all     --->   combines own +
 sensor vector          neighbor messages       neighbor info
 to neighbors           (mean / sum)            into new features
```

This can be repeated for multiple **layers** (rounds):

- 1 layer = info from direct neighbors (vehicles nearby)
- 2 layers = info from 2-hop neighbors (neighbors of neighbors)
- k layers = info from k-hop neighborhood

> More layers = wider "awareness" of the traffic scene

Visual Explanation — Vehicles Exchanging Info

```
Before GNN:                          After 1 GNN layer:

(A)-------(B)                        (A*)-------(B*)
 |  60m    |                          |           |
 |         | 40m     --->             |           |
 |  70m    |                          |           |
(C)-------(D)                        (C*)-------(D*)

Each vehicle knows                   Each vehicle knows about
only its OWN sensors                 itself + neighbors' sensors
```

A* now contains sensor info from A, B, and C

B* now contains sensor info from B, A, and D

> Each vehicle builds a **richer picture** by learning from its neighborhood

Part 4

A Very Simple GNN

Step-by-Step: How a GNN Layer Works

For each vehicle node `i`:

Step 1 — Gather: Collect sensor vectors from neighbor vehicles

$$messages = [sensors_{neighbor_1}, sensors_{neighbor_2}, \ldots]$$

Step 2 — Aggregate: Combine neighbor sensor data

$$aggregated = MEAN(messages)$$

It can be also SUM or MAX

Step 3 — Update: Compute new vehicle representation

$$new_features[i] = W \times CONCAT(features[i],\ aggregated) + b$$

Where `W` is a learnable weight matrix.

The GCN Equation (Simplified)

The **Graph Convolutional Network (GCN)** formula:

$$h'_i = \sigma(W \times MEAN(h_j \; for \; j \in neighbors(i) \cup \{i\}))$$

In plain English:

- Take vehicle `i` and all its neighbor vehicles
- **Average** their sensor feature vectors
- Multiply by a learnable weight matrix **W**
- Apply an activation function **σ** (like ReLU)

That's it! This simple operation is surprisingly powerful.

Aggregation Methods

Different ways to combine neighbor vehicle information:

Method	Formula	Best for
Mean	avg of neighbors	Normalized features
Sum	sum of neighbors	Counting-sensitive tasks
Max	max of neighbors	Salient feature detection

For vehicle prediction, **mean aggregation** works well because:

- It normalizes for varying numbers of nearby vehicles
- Vehicle influence is roughly proportional to proximity

Part 5

GNN for Vehicle Behavior Prediction

How GNN Models Vehicle Interactions

Spatial reasoning:

- GNN captures how **driving behavior spreads** through a vehicle network
- Multiple GNN layers = understanding longer-range vehicle interactions
- Learns which vehicle connections matter most

What the model learns:

- A braking vehicle affects vehicles behind it
- Parallel lanes may compensate for each other
- Intersection vehicles have outsized influence on the network

Spatial Dependencies Between Vehicles

```
Vehicle A (braking suddenly)
  ↓ affects ↓
Vehicle B (behind A) → Vehicle C (behind B) → Vehicle D
```

A GNN with 3 layers can model this entire chain:

- **Layer 1:** B learns A is braking (from A's sensor data)
- **Layer 2:** C learns (through B) that A braked
- **Layer 3:** D has the full picture of the cascade

> The **graph structure** tells the model exactly **how** driving events propagate

Adding the Temporal Dimension

Vehicle behavior is not just spatial — it changes over **time**.

Common approaches:

- Feed multiple **time steps** of sensor data as features
  - e.g., positions at t-5, t-4, ..., t-1, t → predict t+1
- Combine GNN with temporal models:
  - **GNN + RNN** (spatial + temporal)
  - **GNN + Temporal Convolution**

In our demo, we'll use **sensor snapshots as features** — simple but effective!

Part 6

V2X + GNN

How V2X Enhances the Vehicle Graph

Standard vehicle graph:

- Edges = physical proximity only (line-of-sight ~80m)
- Limited to vehicles that are very close on the road

V2X-enhanced graph:

- **Additional edges** from wireless V2X communication (~150m+)
- Vehicles can share data even without direct line of sight
- The graph becomes **richer and more connected**

Adding V2X Communication Edges

```
Standard Graph (80m):        V2X-Enhanced Graph (150m):

  A --- B                      A --- B
  |     |                      | \ / |
  C --- D                      C -X- D
                               |     |
  (only close neighbors)       E --- F
                               (V2X bridges the gaps!)
```

What V2X adds:

- Edges between vehicles within wireless range (~150m)
- Even if no direct line of sight
- Vehicles relay sensor data across the network

Benefits of V2X + GNN

- **Better prediction accuracy**
  - More sensor data flows through the graph
  - The model sees beyond direct physical proximity
- **Faster event detection**
  - Information reaches distant vehicles in fewer hops
  - Earlier warnings, earlier evasive actions
- **Safety improvements**
  - Hazard information propagates network-wide
  - Enables cooperative driving decisions

Part 7

Impact & Applications

Behavior Prediction:

- 10-20% improvement over traditional methods
- Works for speed, trajectory, and risk prediction
- Scales to hundreds of vehicles

Smart Cities:

- Adaptive traffic signal control
- Dynamic speed recommendations
- Cooperative intersection management

V2X Networks:

- Optimal communication topology
- Missing sensor data imputation

Real-World Applications

Application	How GNN Helps
Google Maps	Travel time prediction
Uber	Demand forecasting
Autonomous vehicles	Motion prediction from vehicle graphs
City planning	Infrastructure optimization
Emergency response	Fastest route with live vehicle data

GNNs are already deployed in production systems handling **millions of queries daily**

Part 8

Time to Code!

What You'll Build

Demo 1: Vehicle Prediction with GNN

- Load V2X-Sim-2.0 BEV (Bird's Eye View) data
- Model vehicles as graph nodes with sensor feature vectors
- Create a GCN model with PyTorch Geometric
- Train it to predict vehicle speed
- Visualize the vehicle graph and predictions

Demo 2: V2X-Enhanced Prediction

- Add V2X communication edges (extended range)
- Compare prediction with and without V2X
- See how richer connectivity improves the GNN

The V2X-Sim-2.0 Dataset

V2X-Sim — a multi-agent V2X simulation dataset (NYU AI4CE Lab)

We use the **BEV subset** (~130MB) — Bird's Eye View semantic maps

```
BEV = top-down view of traffic scenes:

    [Car 1]---[Car 2]---[Car 3]
       |                   |
    [Car 4]   [Car 5]---[Car 6]
       |         |
    [Car 7]---[Car 8]---[Car 9]
```

Each vehicle → **node** with sensor features: `[x, y, speed, heading, accel, lidar, camera]`

> Real V2X scenarios, vehicle-centric graphs

Let's Get Started!

Open the Colab notebooks and follow along.

Tips:

- Run cells **one at a time**
- Read the comments — they explain each step
- Ask questions anytime!
- Experiment with the code after the workshop

> **Learning by doing is the best way to understand GNNs**

Thank You!

Questions?

Workshop materials available in the repository.